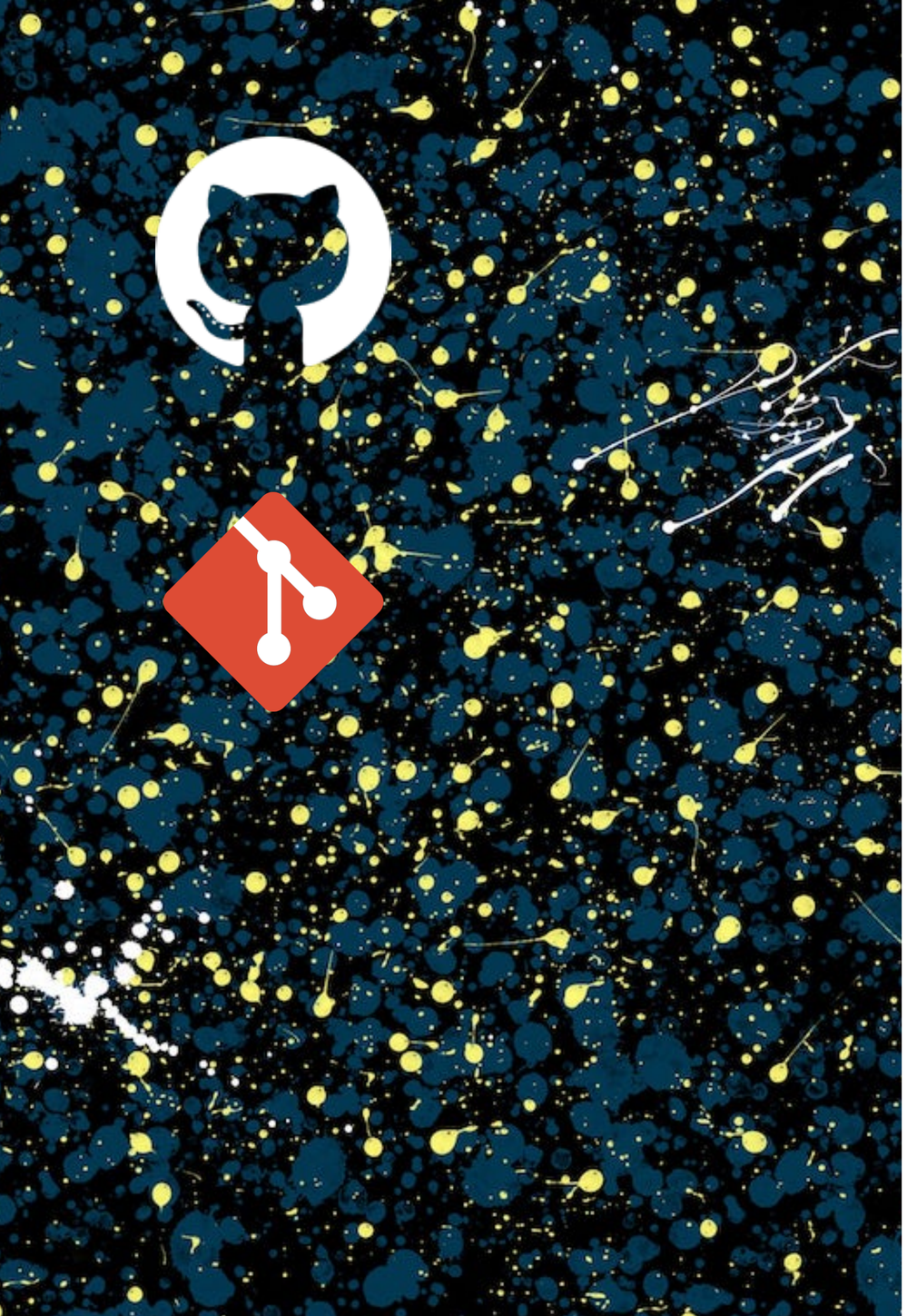
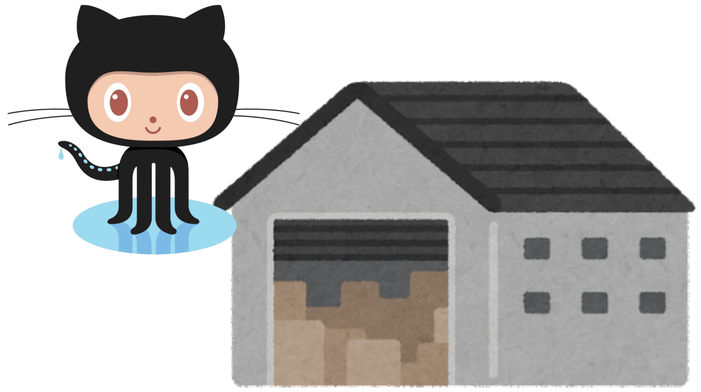
基本・
ブランチ・プルリク



Git/GitHubの流れ

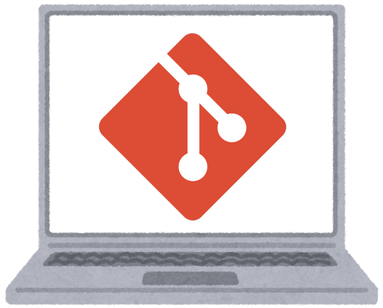
Git/GitHub ざっくり図

Remote



3.アップロード

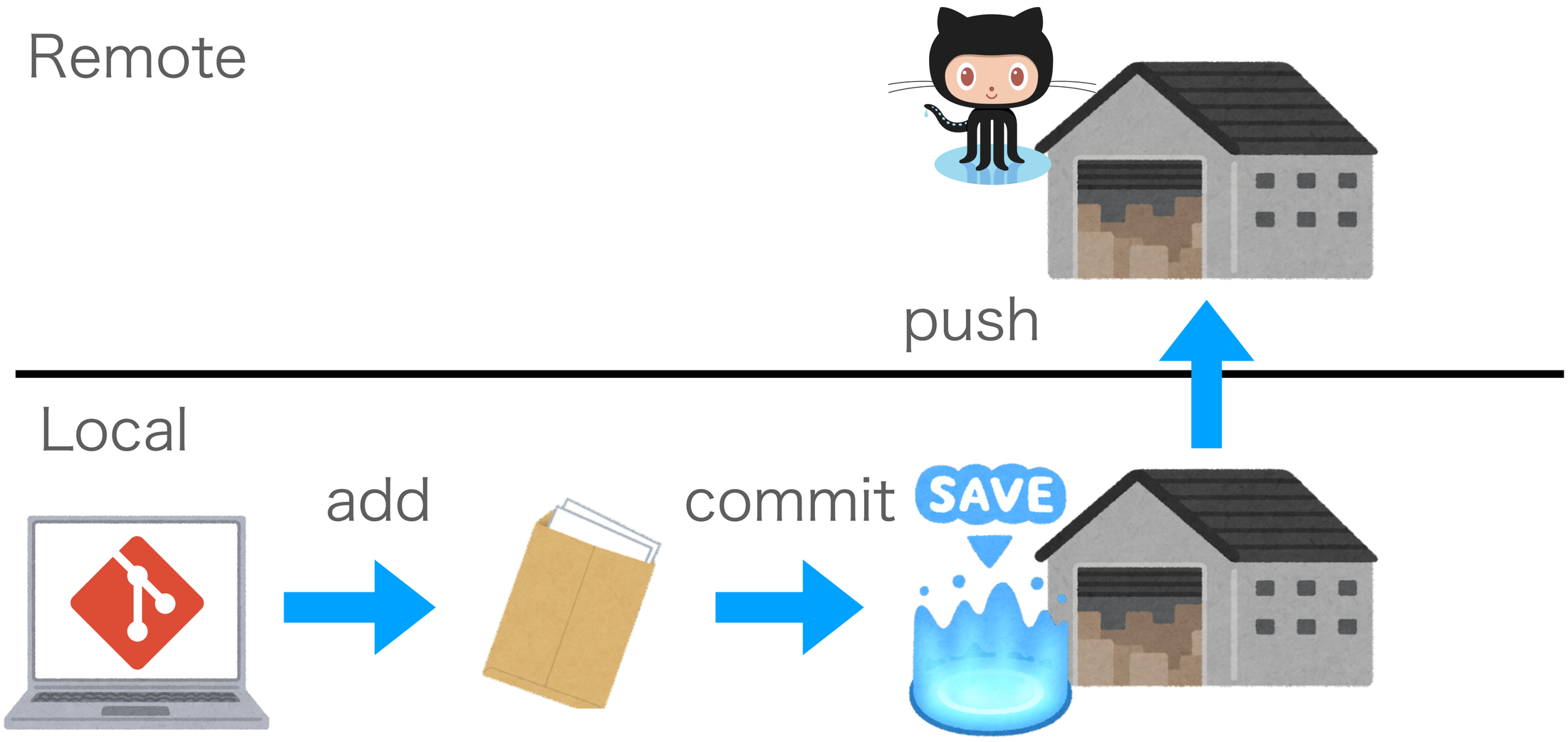
Local



1.追加

2.セーブ

Git/GitHub ざっくり図



Git/GitHubの流れ

add // セーブしたい内容を追加
add . //変更した全てのファイルを追加する

commit // セーブポイント
commit -m "コメント" // コメント

push // アップロード

パソコンでの流れ

1. デスクトップなどどこでもいいので
フォルダを作る (git-sample-udemy)

2. gitのローカルリポジトリ(貯蔵庫)をつくる
git init (初期)

3. コード書く

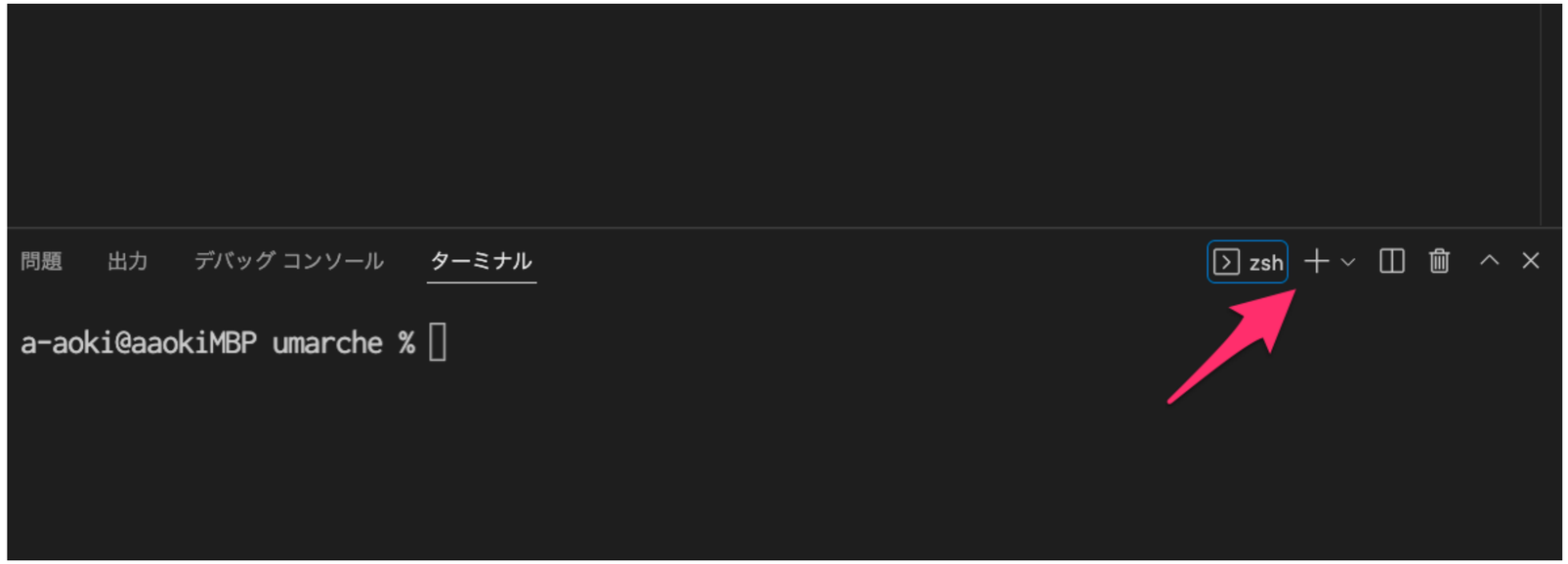
4. add

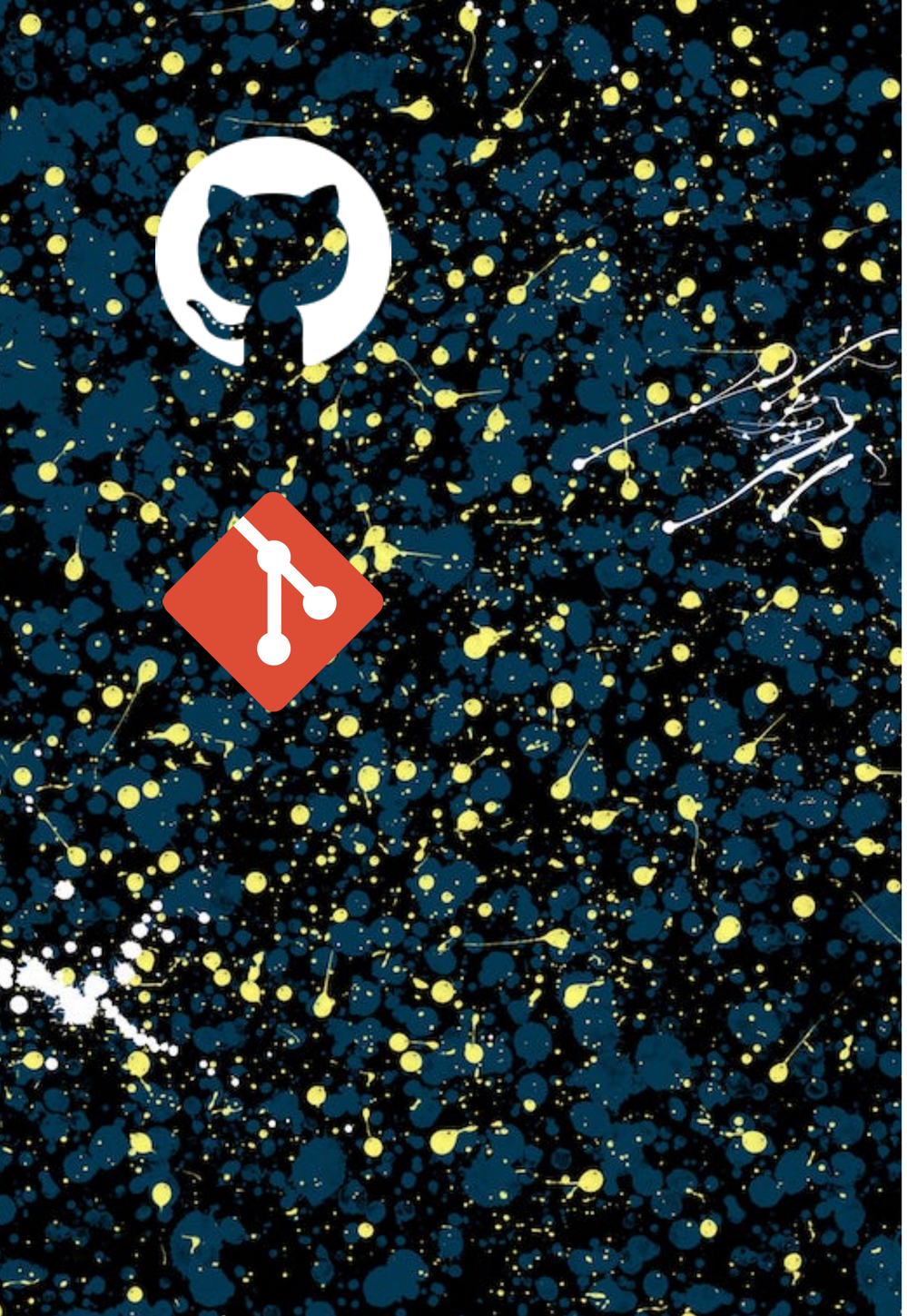
5. commit

Gitのコードを書く場所

ターミナル(mac)かGitBash(win)

VSCodeのターミナル->新しいターミナル
windowsの方はターミナルの+ボタンを押して
GitBashを選んでください





GitHubで
リモートリポジトリを作成

Git/GitHub ざっくり図

GitHubでリモートリポジトリ

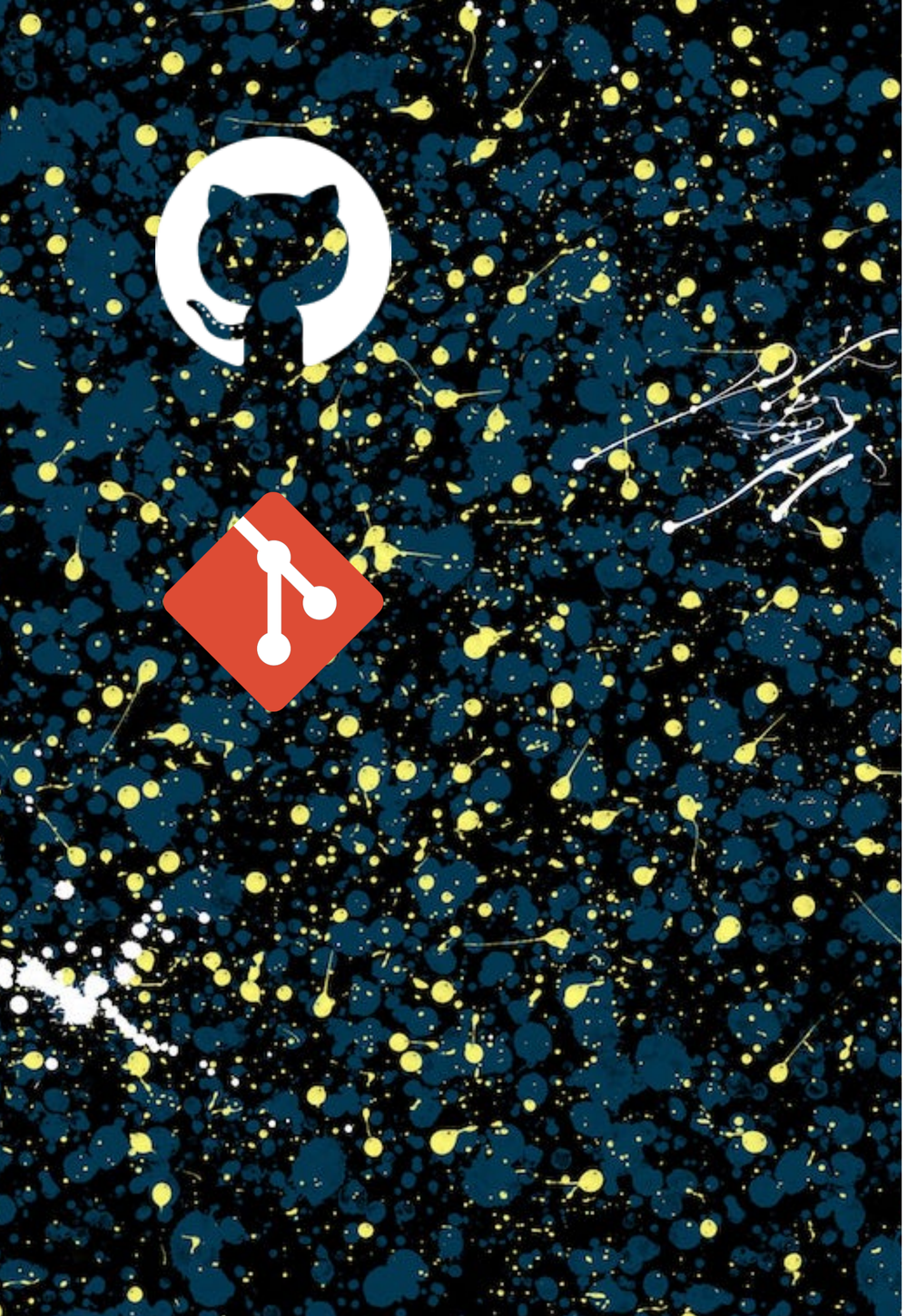
git-sample-udemy などリポジトリを作成

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:ユーザー名/git-sample.git
git push -u origin main
```

Branchの名前の補足

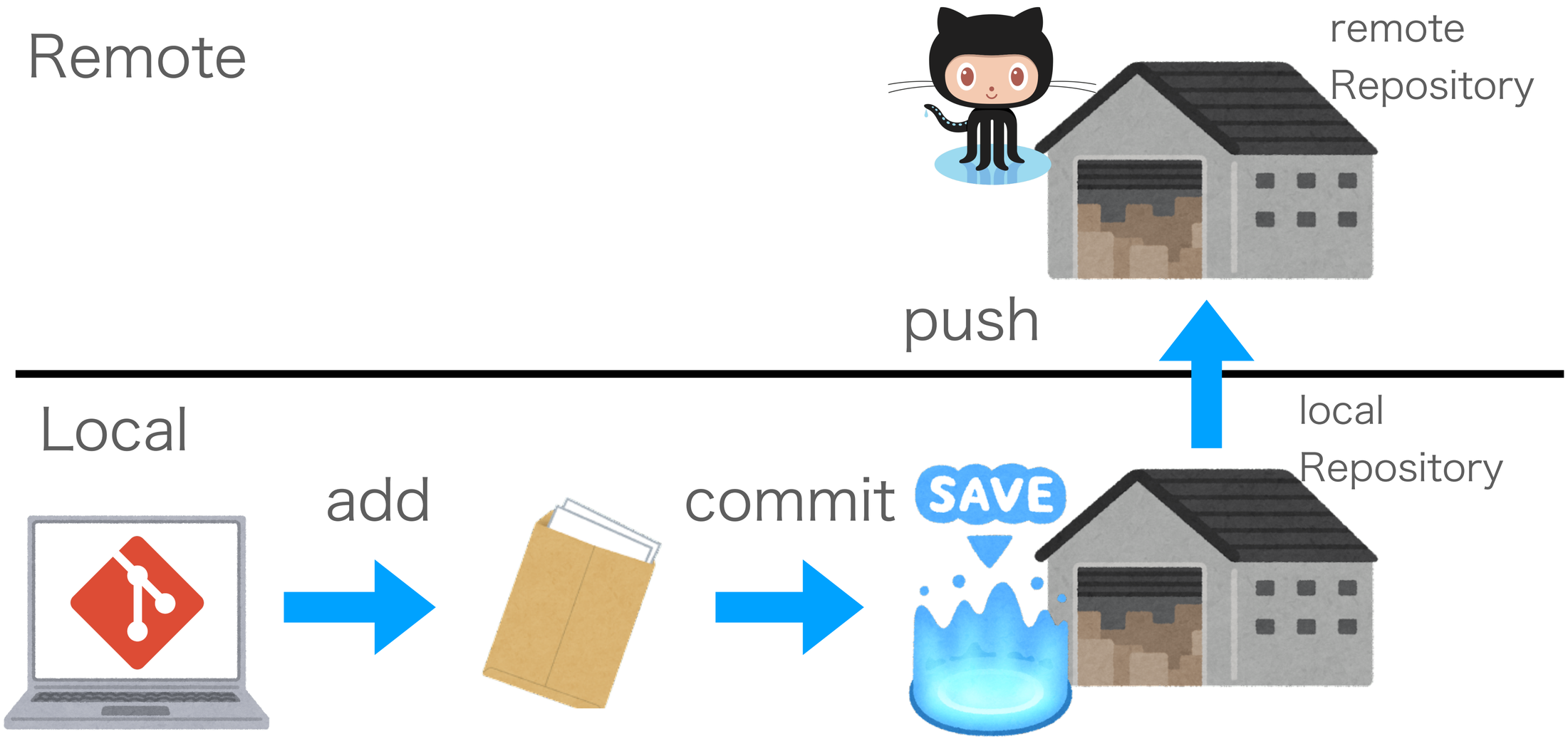
IT業界では昔から
master/slave という言葉が使われていた。

slaveは奴隷という意味もあり、
アメリカで黒人人権運動(BLM)が起こった後
main/sub という言葉に変わりつつある。



2回目以降の流れ

Git/GitHub ざっくり図

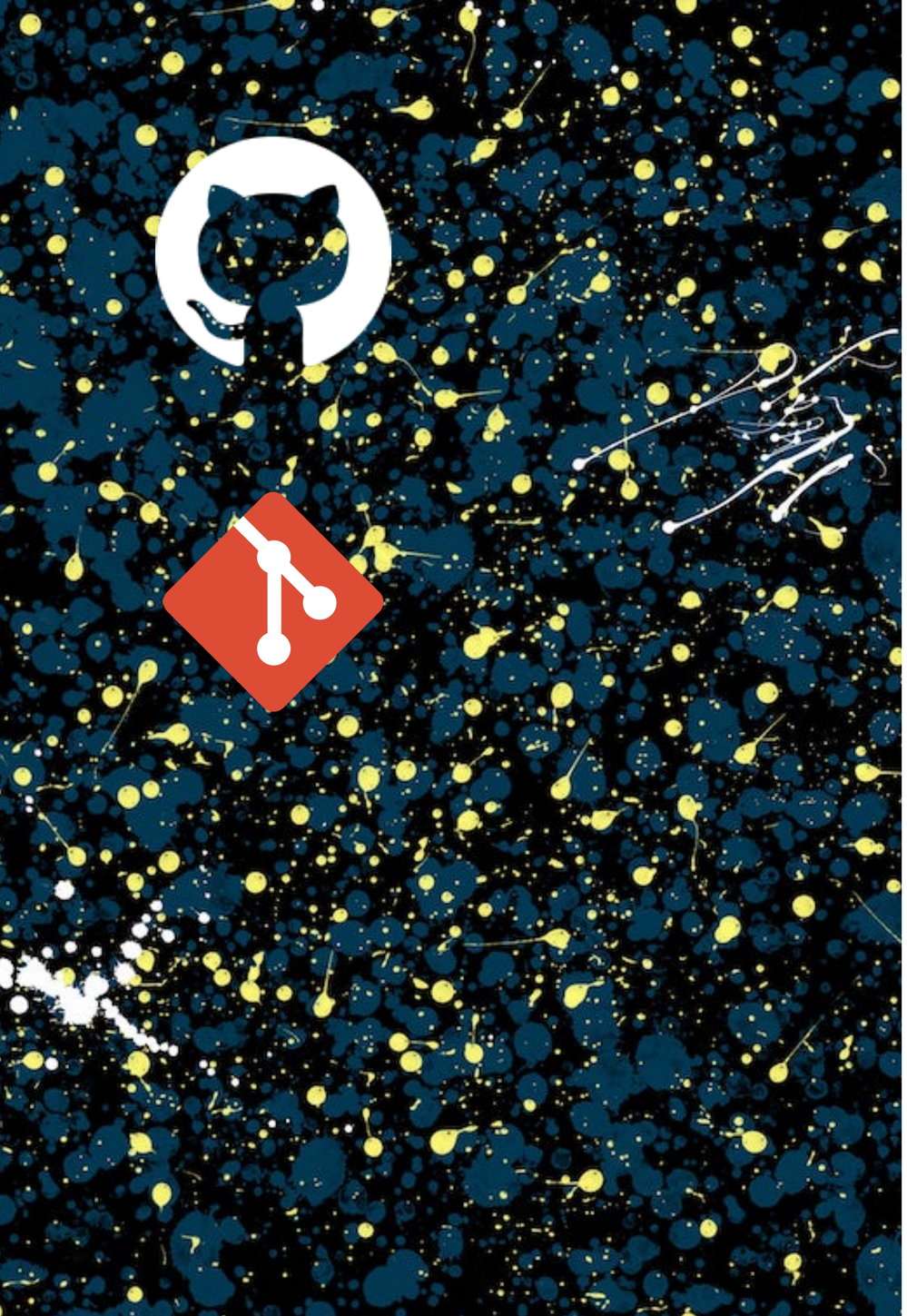


2回目以降は3つのコマンド

```
git add .
 // VSCode上で追加ファイルも見れる

git commit -m "コメント"
 // VSCode上GitGraphでIDも見れる

git push origin main
//        リモート名 リモートブランチ
```



これまでの補足

補足1 macで毎回パスを聞かれる

GitHubの認証を
パスワード認証->トークン認証に 移行

git-credential-osxkeychain　という
パスワード入力を省略できる機能が使われていてその情報が残っている

スポットライトで keychain access と入力し右クリックで削除

補足2 git push と git push -u

-u は --set-upstream と同じ
upstream(上流)

オプションを付けると
ローカルリポジトリの現在ブランチ(main)の上流を
origin/main に規定

オプションあり / なし
git push / git push origin main
git fetch / git fetch origin main
git pull / git pull origin main

git push と git push -u

上流ブランチの設定確認
git branch -vv

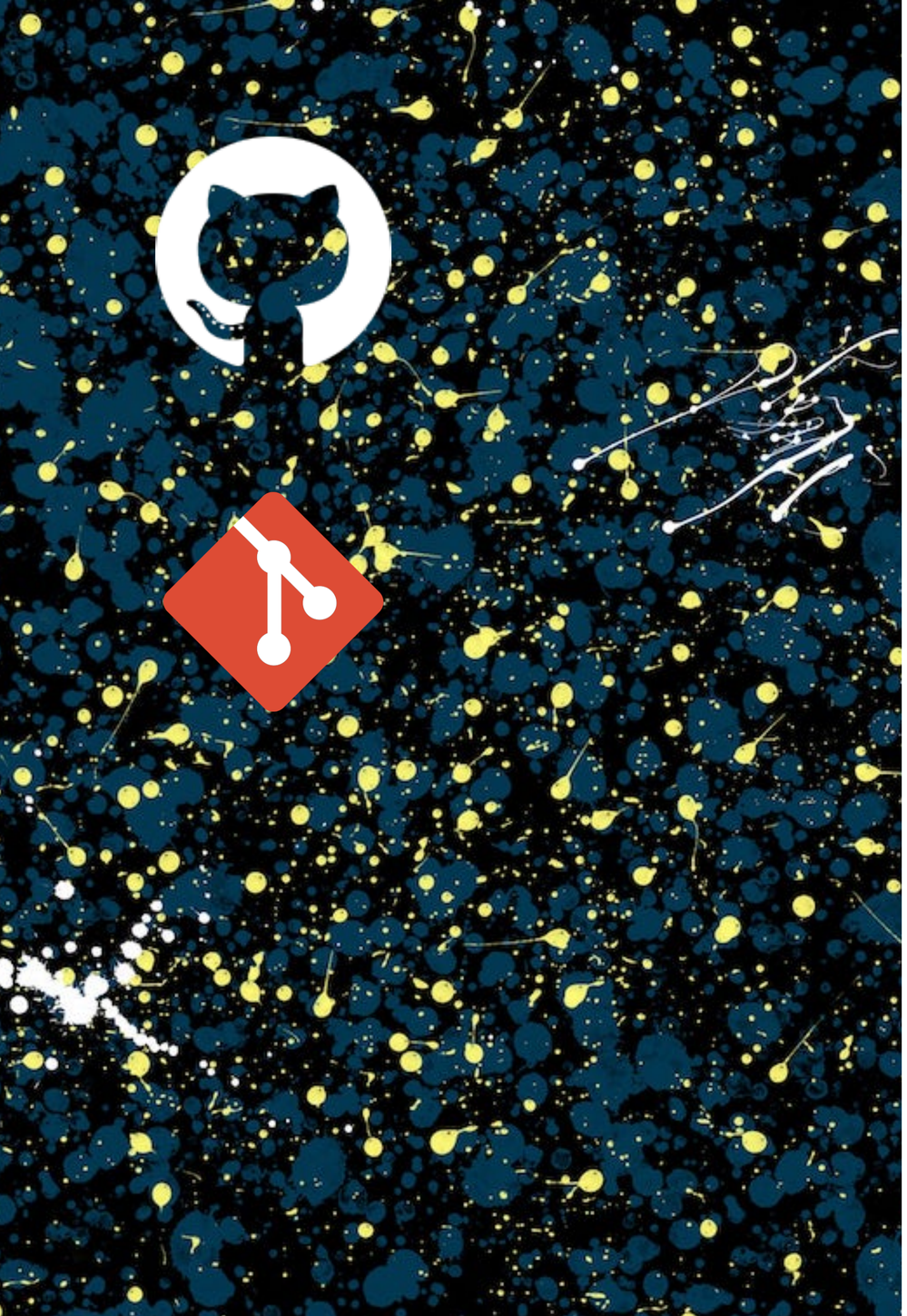
上流ブランチの変更
git branch -u リモートリポジトリ名/ブランチ名

push時に気をつける事

.gitignore

GitHubにアップしたくないファイル・フォルダを書いておく
.env など

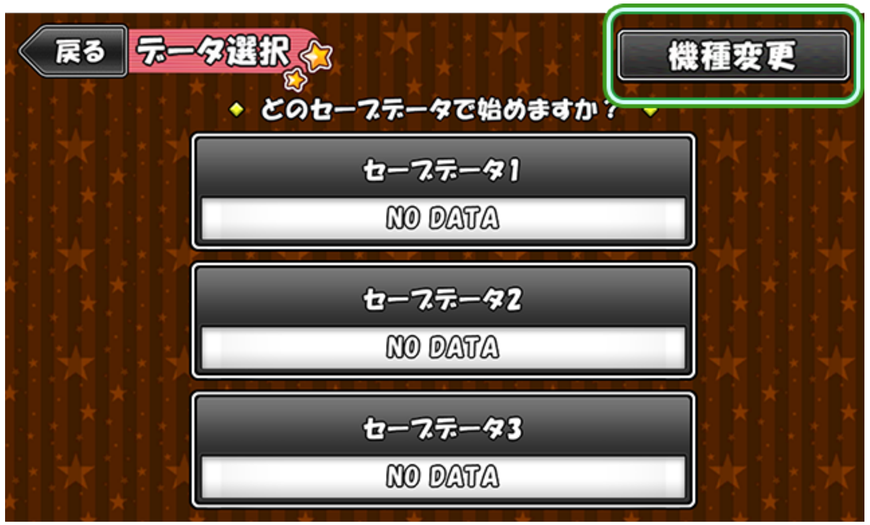
.env_example などをつくりダミーパスワードをいれておくと
他の人もファイルが必要だと気づくことができる

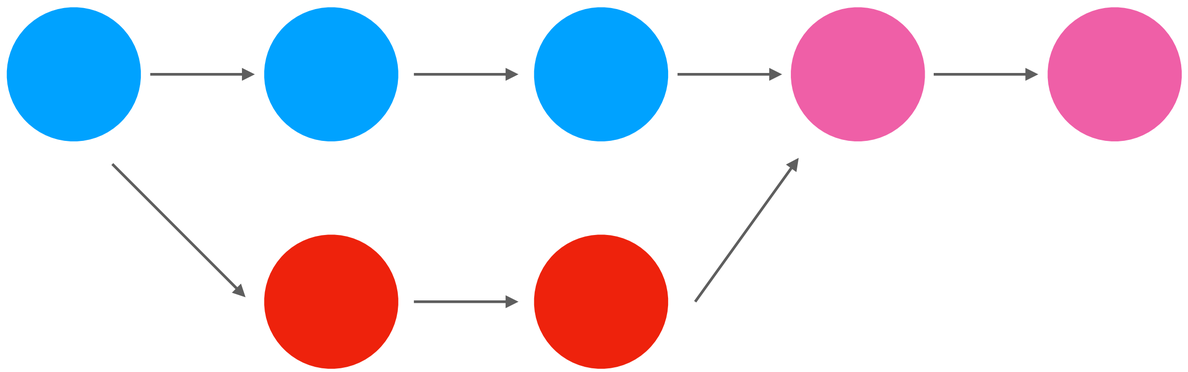


ブランチ

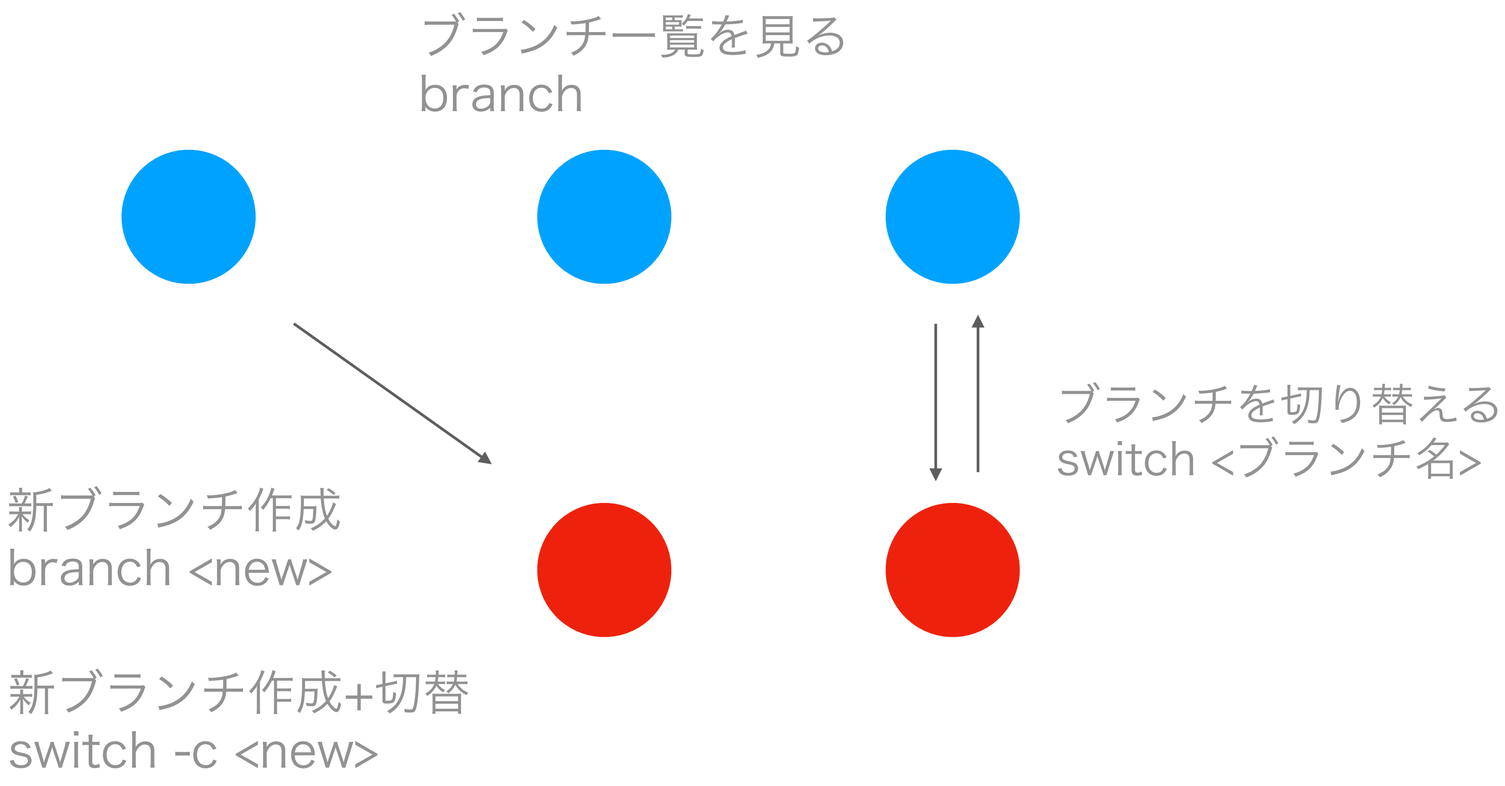
git branch(ブランチ)

セーブデータのように
途中でコードの進行を分岐できる
さらに後でマージ(結合)もできる





git branch(ブランチ)



git branch(ブランチ)

git branch // ブランチの確認

git branch <new> // ブランチの作成

git branch -m <old> <new> // ブランチ名変更

git switch <ブランチ名> // ブランチの切替

git siwtch -c <ブランチ名> // ブランチ作成 + 切替

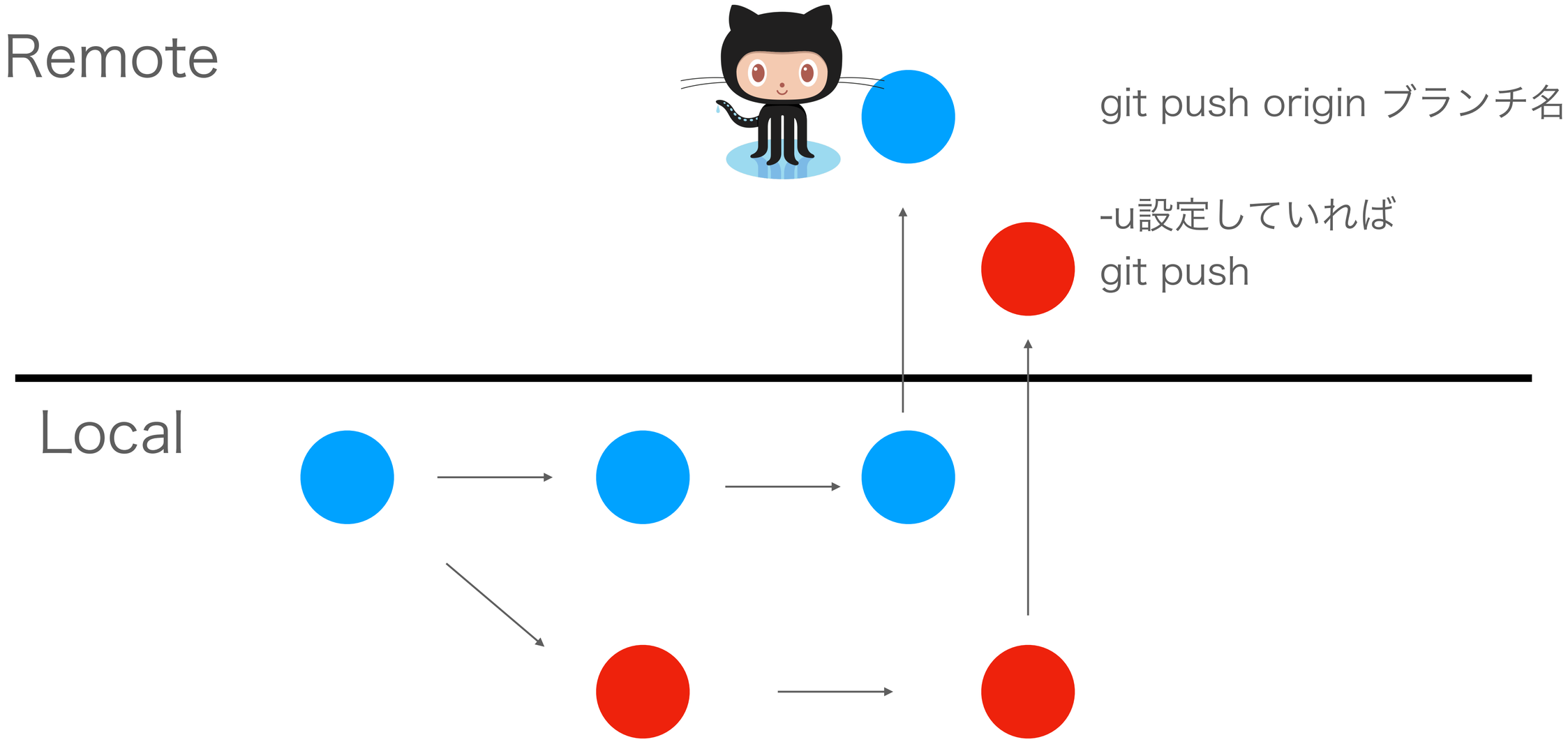
git branch -d <ブランチ名> // ブランチの削除

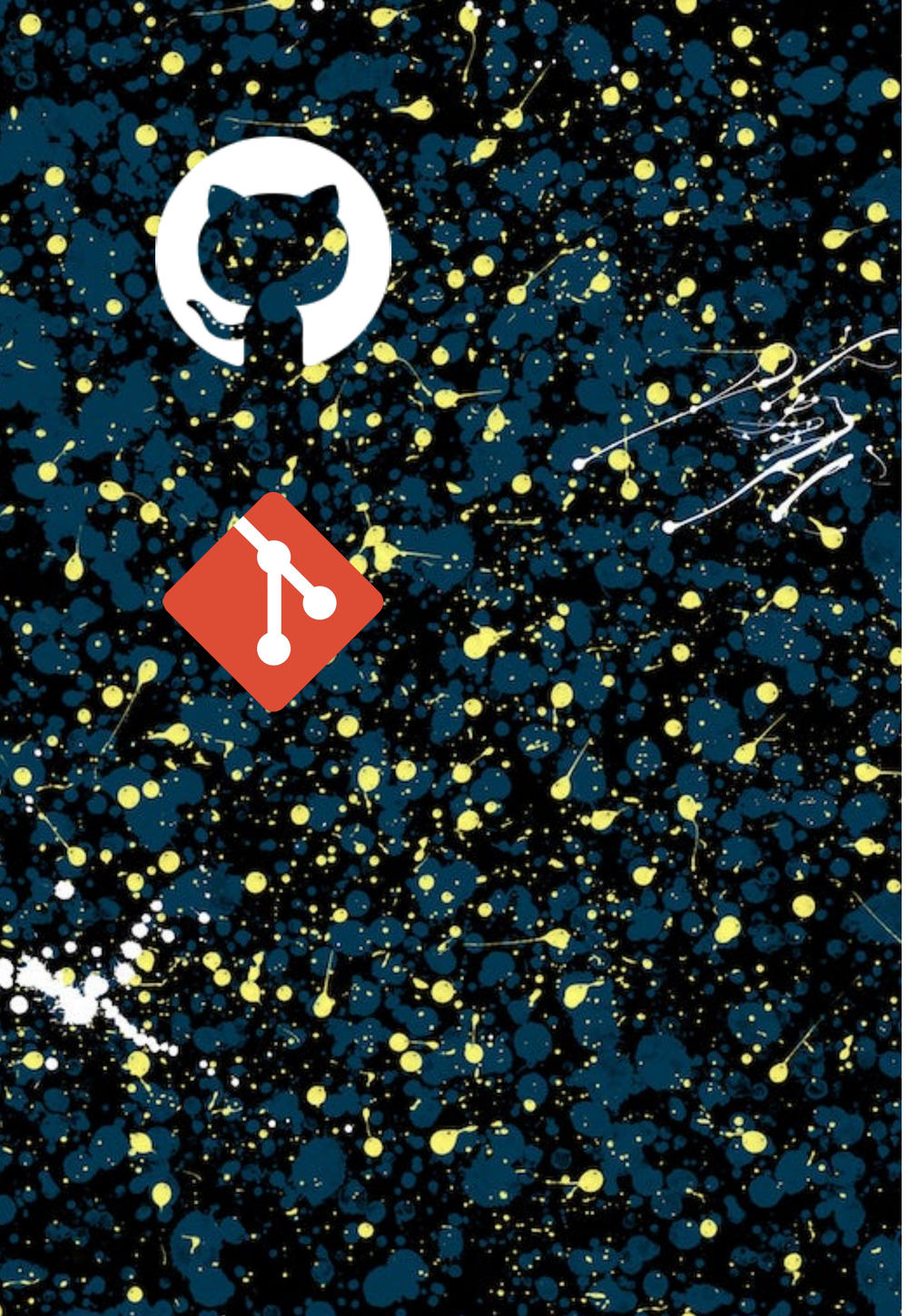
git branchの補足

昔は
checkout でブランチ切替

機能が多すぎるという事で
switch と restore の新コマンドが追加された

異なるブランチで進めてみる





プルリクエスト
(Pull Request)

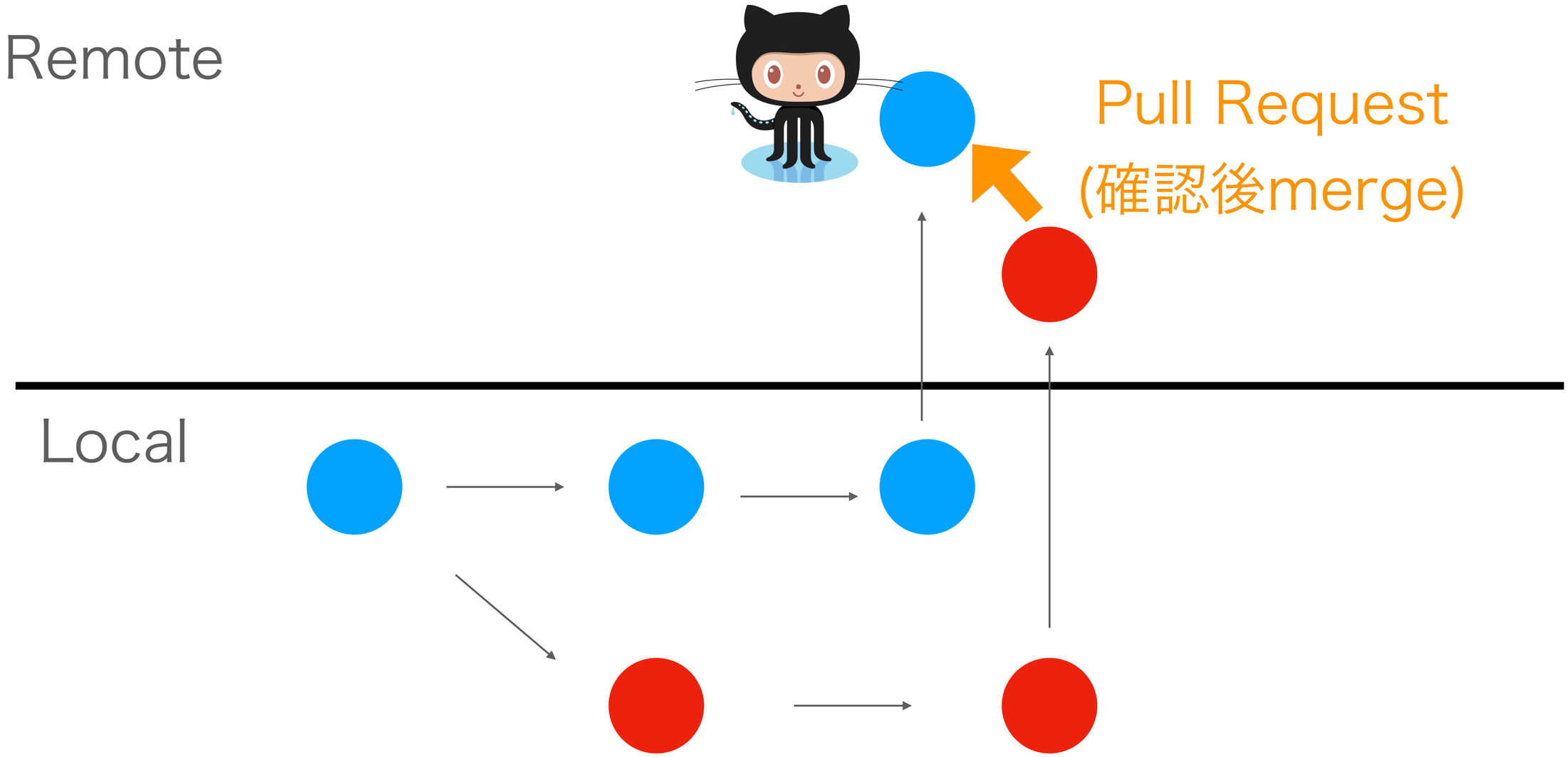
プルリクエスト(プルリク)

GitHubの機能

・異なるブランチを合わせたい(merge)
・コードがぶつかっていないか自動チェック(コンフリクト)
・mergeする前にコードチェックしたい
・確認した内容をコメントしたい
・確認okなら合わせる(merge)

などをまとめた機能

プルリクはGitHub の機能



プルリクの流れ

1-1.新ブランチ compare & pull Request ブランチ比較

1-2. 既存ブランチ (黄色背景は表示されないのでメニューのpull request -> new pull request)から作成

2. create pull Request プルリク作成

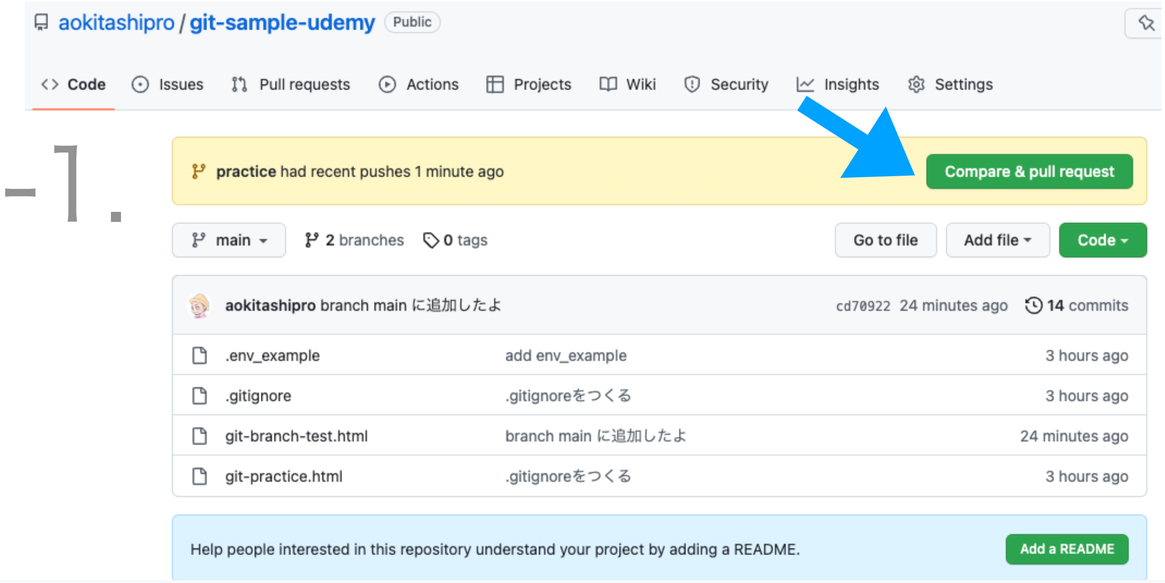
3. コード比較され問題なければ Merge Pull Request をクリック

4. Confirm merge をクリック

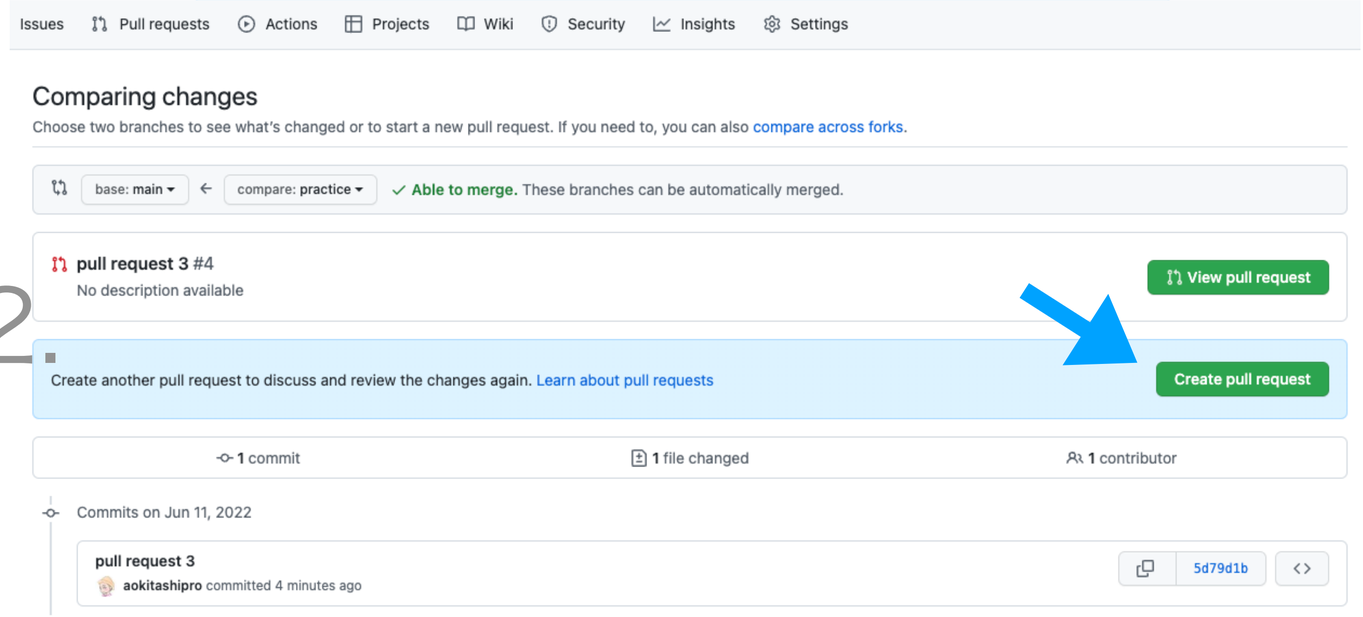
5. プルリクがマージ&クローズされる

プルリクの流れ (キャプチャ)

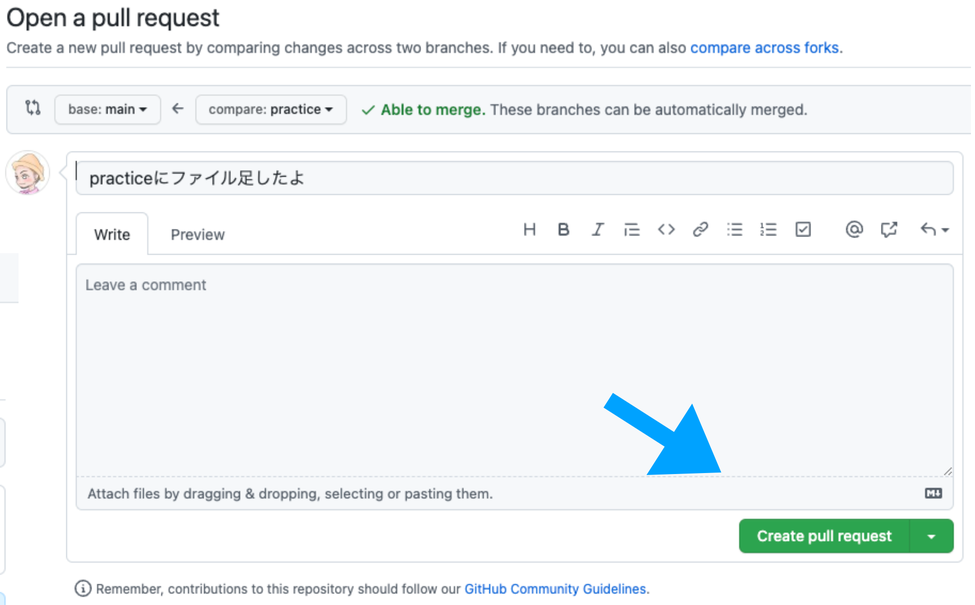
1-1.



1-2.

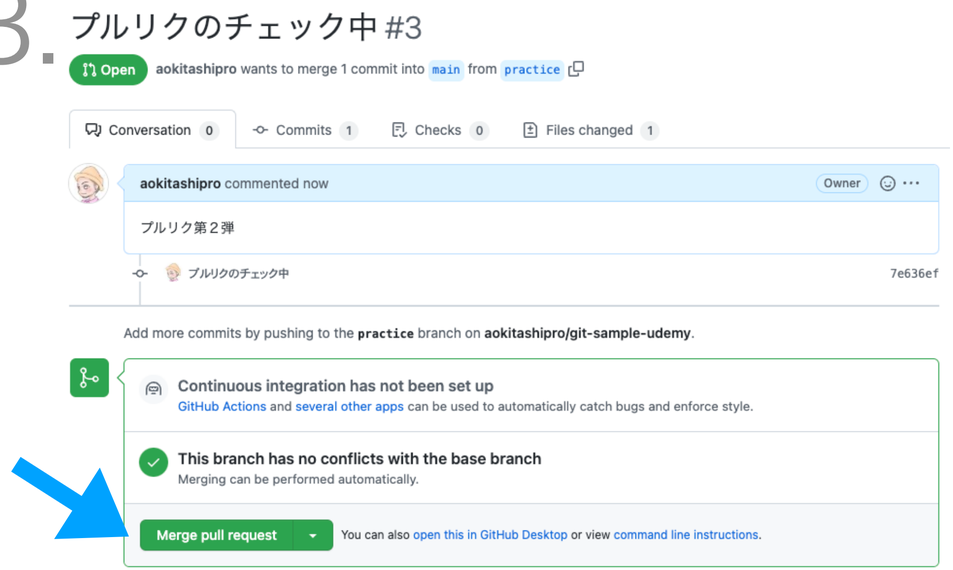


2.

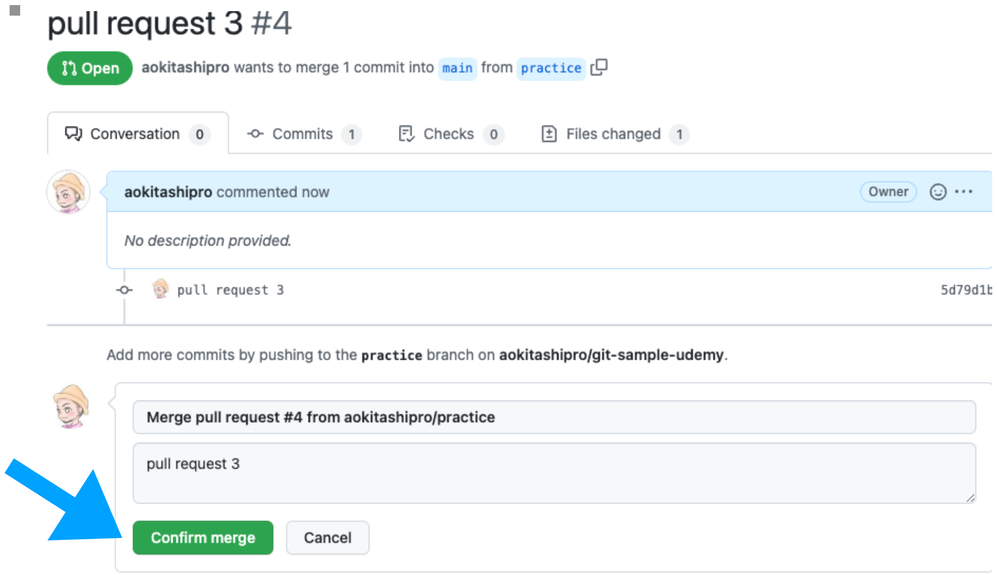


プルリクの流れ (キャプチャ)

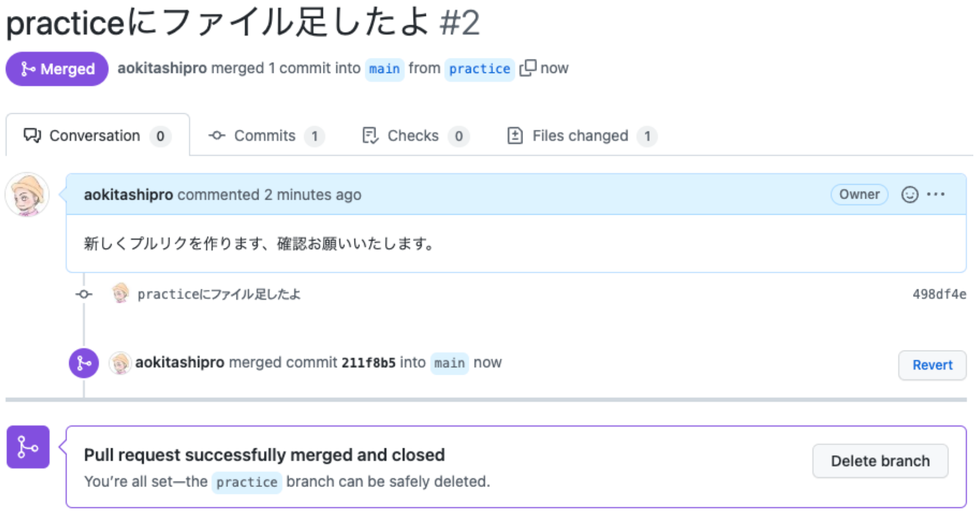
3.



4.

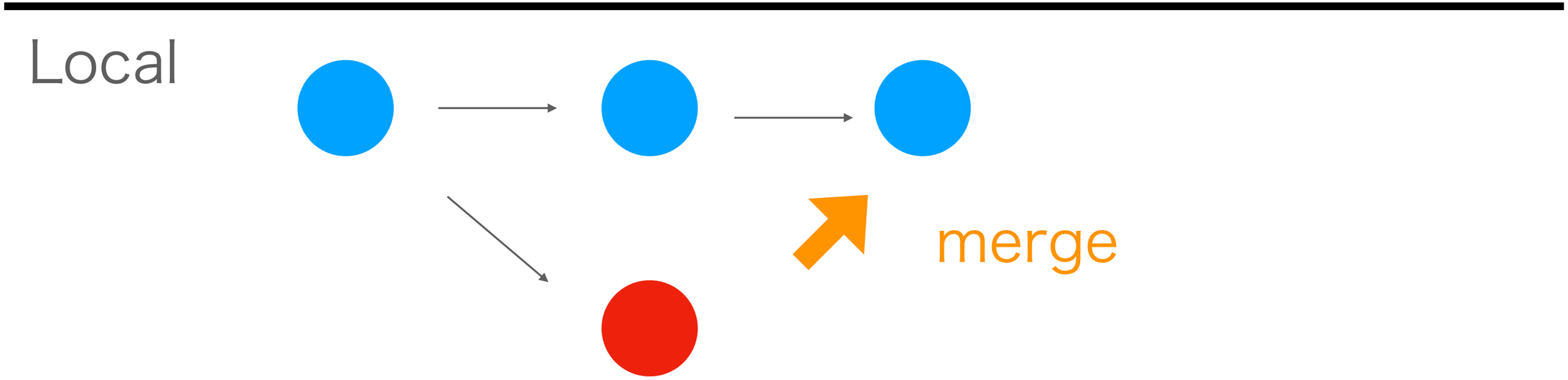


5.



ローカル側でマージ

git merge マージしたいブランチ名



ローカルマージでコメント

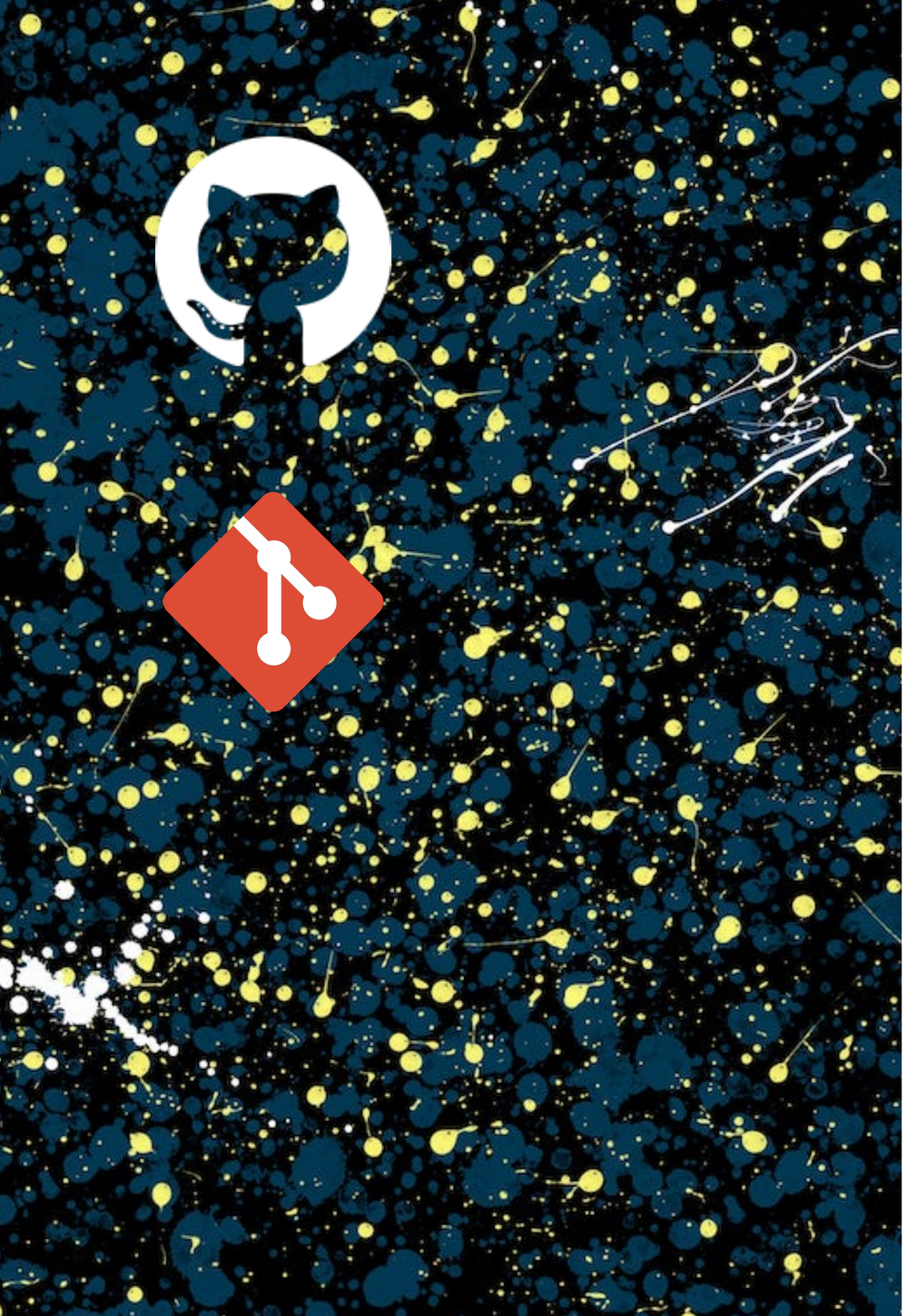
ローカルマージ時
viエディタでコメントをもとめられる場合があります

```
Merge branch 'practice'
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
~
```

ローカルマージでコメント

viエディタは2つモードがあります
Escキーを押すとコマンドモード
iかaを押すと挿入(INSERT)モード

コメントなしなら :q でviエディタ終了
コマンドしたら、コマンドモードに戻り
:wq と入力すればokです。 (上書き保存して終了)



補助系のコマンド

補助系のコマンド その1

git status // コミット前のファイル確認

git log // 全コミットログ
git log --oneline // 1行
git log --graph // グラフ表示

git config -l // 設定確認

git --version // バージョン確認

補助系のコマンド その2

git diff // 前コミットからの変更を確認

git diff <ブランチ名> <ブランチ名>
// ブランチ間の差分

git diff <commit_id> <commit_id>
<name_of_file> // コミット間の差分